Space Defender Agent Gameplay Using Q-Learning

Youtube : https://youtu.be/eAuSDEHph-A
Github: https://github.com/zhoubojian-stevenchow/cds524_assignment_1_Space_Defender_by_Dueling_DQN_and_PER
All Files of This Project:https://drive.google.com/drive/folders/1h80pAkvDW9Vynax10PZrVCs2lG2-UoBe?usp=sharing

Introduction

The game, designed for this project, is the classic shooter game called Space Defender, in which the player will control a flight to shoot enemy and move horizontally at the bottom of the screen to target enemy or dodge enemy. This report is written to demonstrate my progress in adopting Q-learning with a neural network called Deep Q-Network (DQN) [1] to train a agent to play this game. More specifically, this DQN adopts a more advanced architecture, the Dueling Deep Q-Network [2], which is expected to boost the learning process of the agent and train the agent to be more strategic when making its moves in the gameplay. Meanwhile, a SumTree Class is adopted to implement Prioritized Experience Replay [3], which is designed to mitigate the influence of bias caused by the training process itself. This report documents the game design, Q-learning implementation, evaluation methodology, and the challenges encountered during development.

2. Game Design

2.1 Objective and Rules

The objective of game is demonstrative. In short, it consists of two parts, killing as many enemy as the player can during the game and surviving through time as long as possible. The player ship has 3 health points and loses one point when hit by an enemy bullet or when colliding with an enemy ship. The game ends when health reaches zero. Each enemy destroyed awards 100 points, with the goal of maximizing the total score.

Game Rules: (1) The player ship can move horizontally and shoot vertically upward at the bottom of the screen. (2) Enemies spawn from the top at regular intervals (every 60 frames) and move downward with slight horizontal drift, while shooting bullets downward to the bottom of the screen. (3) There can be no ore than 8 enemies show up on the screen for each frame of the

game. (4) Enemies will fire bullets randomly in each frame by the chance of 2%. (5) The player has a 15-frame cooldown between shots. (6) For each gameplay, the player has 3 health points, which will be deducted one point each time it collides with an enemy or gets hit by enemy's bullets. (7) The player will lose the gameplay every time its health point hits 0. (8) Destroying an enemy by either collision or shooting will result in gaining a 100 points reward.

2.2 State Space

The state representation is critical for effective learning. I designed a 22-dimensional feature vector capturing the essential game information, then stack 4 consecutive frames to provide temporal context, resulting in an 88-dimensional state vector.

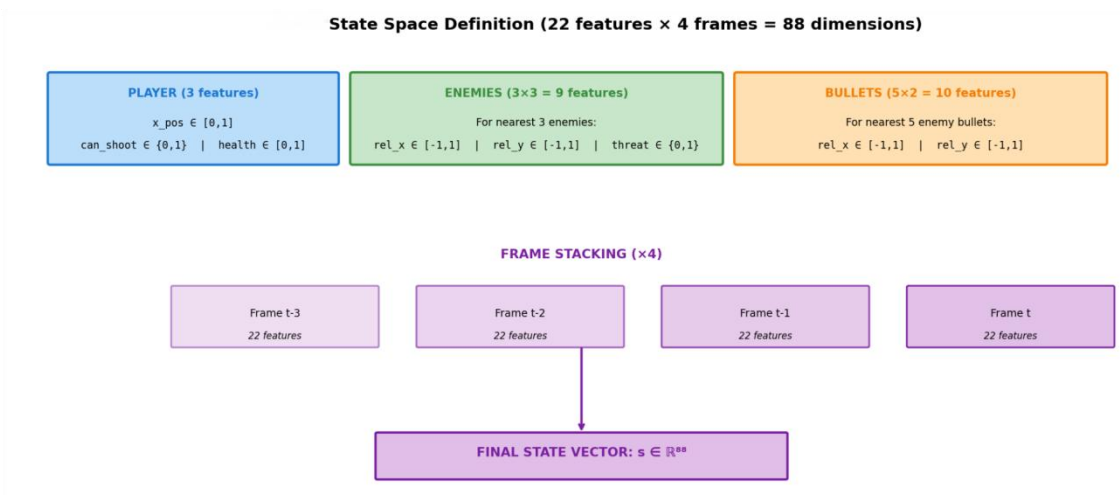

Figure 1: State space definition showing the 22 features per frame and 4-frame stacking

The state includes: player position (normalized x-coordinate), shooting availability (binary), health status (normalized), relative positions and threat levels of the 3 nearest enemies, and relative positions of the 5 nearest enemy bullets. This design provides the agent with sufficient information to make informed decisions about movement and shooting.

2.3 Action Space

The action space consists of 6 discrete actions that cover all meaningful combinations of movement and shooting, including "Move Left", "Move Right", "Stay", "Shoot", "Move Left And Shoot" and "Move Right and Shoot":

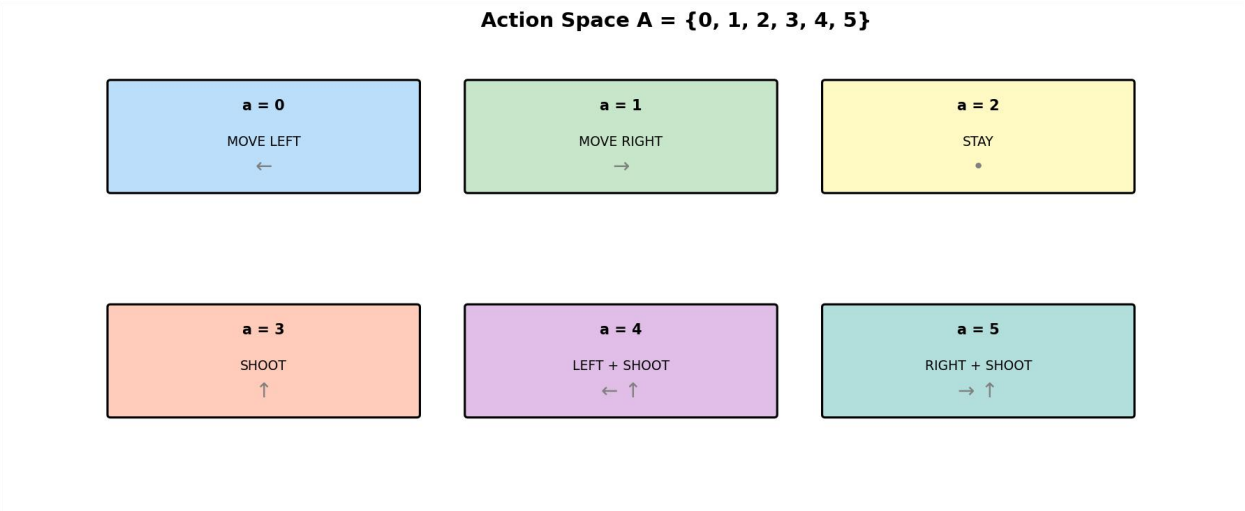

Figure 2: 6 different actions

2.4 Reward Function

A key insight from my iterative development (versions 1-5) was that complex reward shaping often leads to agents exploiting the reward signals rather than learning the intended behavior. Version 7 adopts a minimalist philosophy: only reward the core objectives.

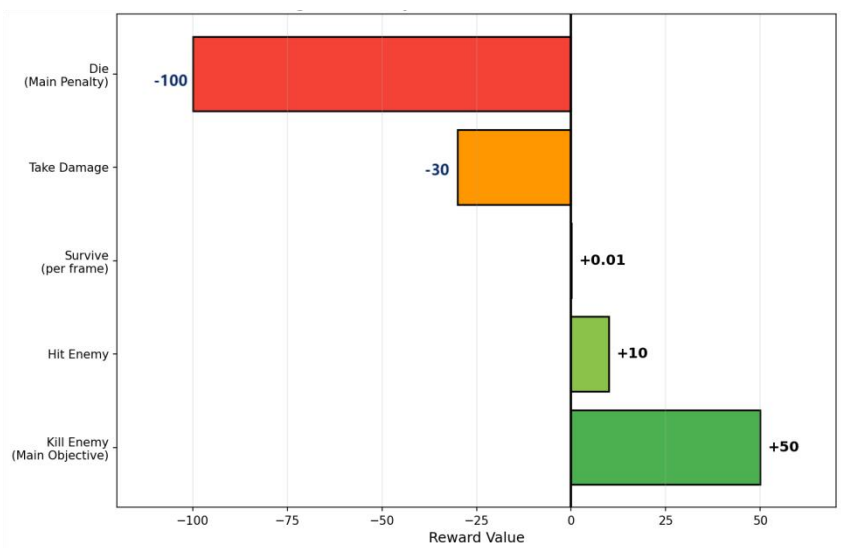

Figure 3: Simplified reward structure focusing on core objectives

This simplified design eliminates positioning rewards, escape bonuses, and camping penalties that plagued earlier versions. Instead, the reward structure only focus on killing enemy effectively (by granting reward of 50 points for killing enemies and 10 points for hitting enemy)

and surviving through the game for a longer run (by granting reward of 0.01 point of surviving each frame and getting punished from taking damage with 30 points deduction and dying with 100 points deduction).The agent must discover effective strategies purely from the consequences of killing enemies and taking damage.

2.5 UI Design

The UI design of this game follows the classic version of Space Defender games, with a dark back ground as cosmetic background, while simplifies the enemy into a red downward triangle and the player's fighter jet into a blue arrow with an orange triangle flame at its rare. At the top left corner, it shows the score for killing enemy, the number of enemy being killed and the lives left counted as the green squares. At the top right corner, it shows the model version used in this gameplay. At the bottom left corner, it shows the current actions taken by the agent and the current reward gained in the game. At the bottom right corner, it shows the Q-value rendered by the model for the 6 different actions based on the lasted 4 frames received from the current game.

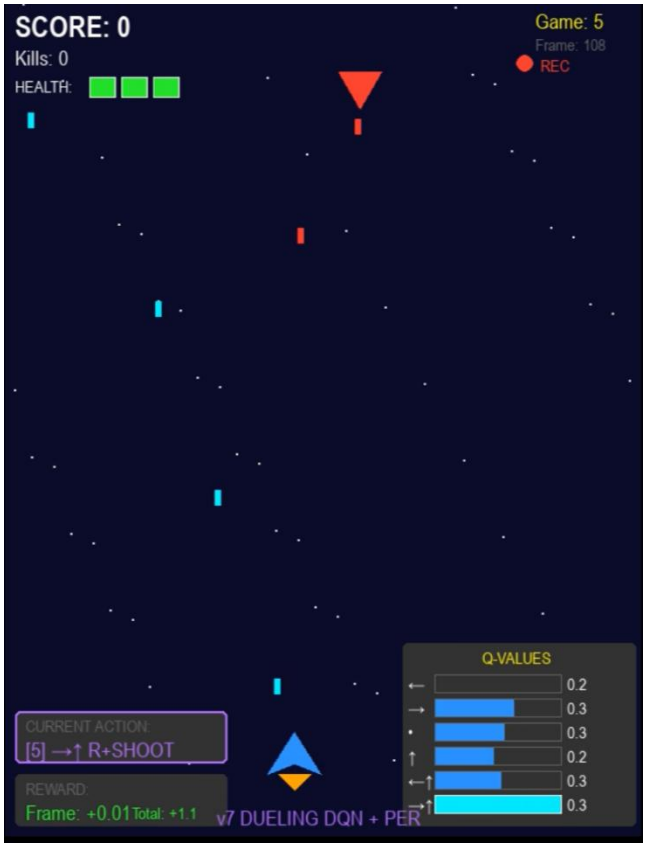

Figure 4: UI Design(Game Start)

Figure 5: UI Design(Game Over)

3. Q-Learning Implementation

3.1 Algorithm Overview

Q-learning is a model-free reinforcement learning algorithm that learns an action-value function Q(s,a) representing the expected cumulative reward when taking action a in state s and following the optimal policy thereafter [1]. The core update rule follows the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max Q(s',a') - Q(s,a)]$$

where α is the learning rate, γ is the discount factor, and r is the immediate reward.

3.2 Dueling DQN Architecture

I implement a Dueling DQN architecture that separates the value function V(s) from the advantage function A(s,a). This decomposition helps the network learn which states are valuable independent of actions, accelerating learning in states where action choice matters less [2] .

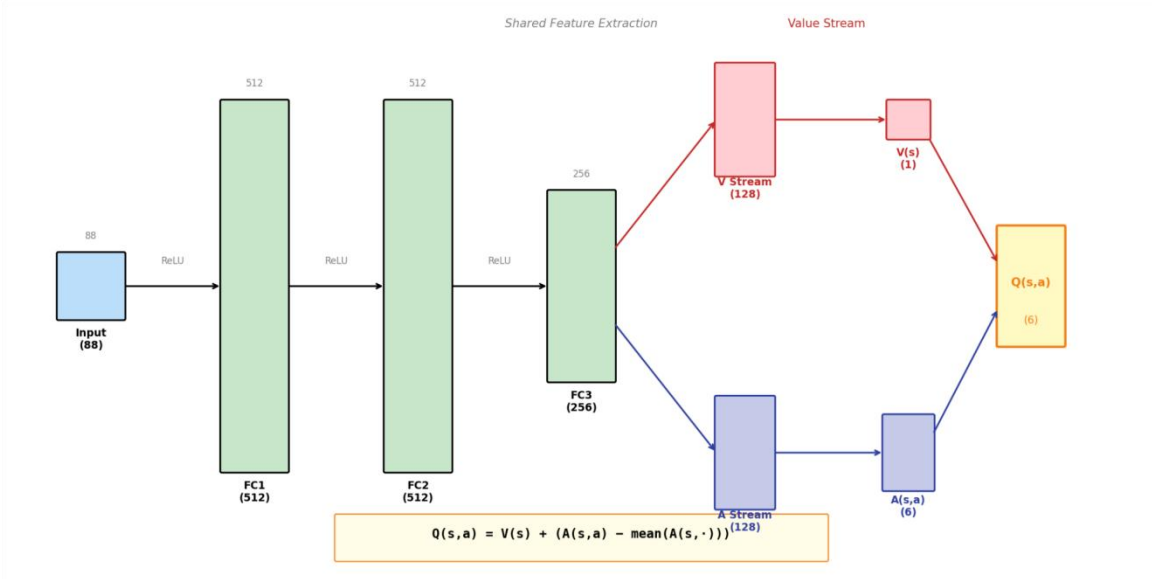

Figure 6: Dueling DQN architecture with shared feature extraction and separate value/advantage streams

The network contains approximately 430,000 trainable parameters with shared feature extraction layers (512→512→256) feeding into separate value (128→1) and advantage (128→6) streams. The final Q-values are computed as: $Q(s,a) = V(s) + (A(s,a) - \text{mean}(A(s,\cdot)))$.

3.3 Prioritized Experience Replay

Standard experience replay samples transitions uniformly, but not all experiences are equally informative. Prioritized Experience Replay (PER) samples transitions proportional to their TD-error, ensuring that surprising or important experiences (like deaths or kills) are replayed more frequently [3]. I use a SumTree data structure for O(log n) sampling and importance sampling weights to correct for the bias introduced by non-uniform sampling.

3.4 Hyperparameters

Key hyperparameters were tuned through extensive experimentation: learning rate $\alpha=0.0001$, discount factor $\gamma=0.99$, epsilon decay from 1.0 to 0.02 with factor 0.9995, batch size 128, memory size 200,000, target network update every 500 episodes, PER $\alpha=0.6$, and β annealed from 0.4 to 1.0.

4. System Architecture

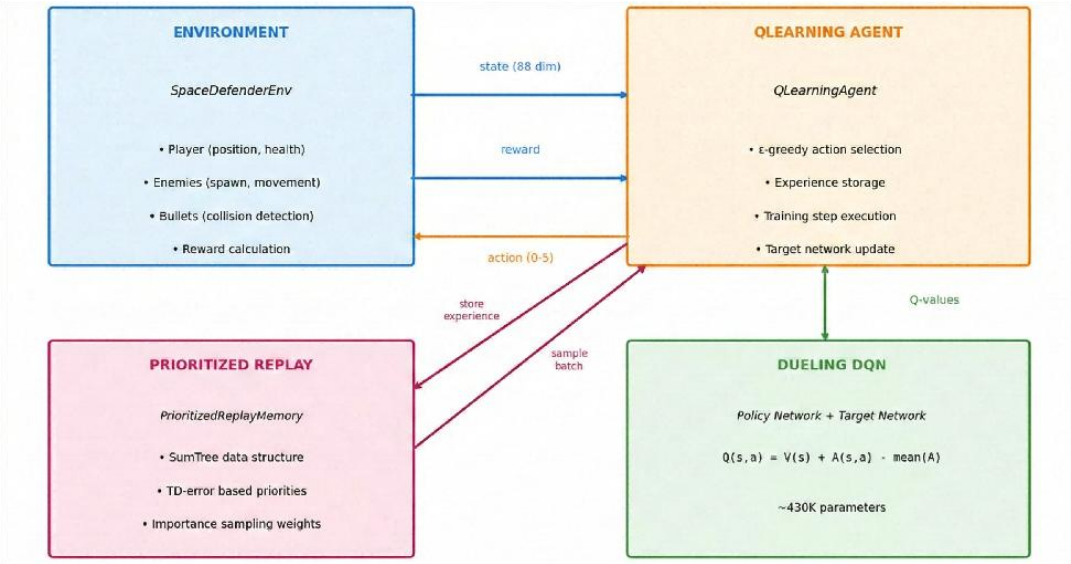

Figure 7: Overall system architecture showing component interactions

The system consists of four main parts that interact with each other, including Environement setup, the QLearning Agent, the Dueling DQN networks and the Prioritized Replay Memory. More specifically, in the Environment setup called SpaceDefenderEnv, the QLearning Agent is powered by its brain, which is the Dueling DQN networks, to decide each action to take based on the Q-value generated in the Dueling DQN networks. Meanwhile, the QLearning Agent will store the errors it made and the experience it learn in the Prioritized Replay Memory, which will help the agent to acquire strategic insight during the gameplay.

5. Evaluation Results

The agent is evaluated with the following metrics: mean evaluation score, standard deviation, score range (min/max), average kills per game, and average survival time.

Key Results: The simplified reward structure in v7 demonstrated superior learning stability compared to earlier versions with complex reward shaping. The agent learned emergent behaviors including: strategic positioning to create firing lines, bullet dodging through anticipatory movement, and prioritization of closer threats.

6. Challenges, Solutions and Limitation

6.1 Reward Hacking

Challenge: In version 1 to 5 of this project, instead of learning to actually kill enemy, the agent mostly learns to dodge bullets by exploiting the intermediate rewards. It is because of the complex reward structures of version 1 to 5, including intermediate rewards such as positioning bonuses, escape rewards, and camping penalties.

Solution: Version 7 simplifies the reward structure to only 5 signals directly tied to core objectives. This forced the agent to discover effective strategies rather than exploit reward loopholes.

6.2 Sample Efficiency

Challenge: Important transitions (deaths, kills) were rare compared to normal gameplay frames, leading to slow learning of critical behaviors.

Solution: Prioritized Experience Replay ensures high-TD-error transitions are sampled more frequently, dramatically improving sample efficiency for learning from deaths and successful kills.

6.3 Limitation

Even though the issue of high variance in performance in the version 1-5 has been significantly mitigated in version 6 and 7 by adopting the Dueling DQN, the variance in performance should occasionally appear, with good policies sometimes degrading during continued training.

7. Conclusion and Future Works

This project successfully demonstrates the application of advanced Q-learning techniques to an action game environment. The Space Defender agent learns complex behaviors including enemy targeting, bullet avoidance, and strategic positioning entirely through trial-and-error learning with minimal reward engineering.

As for the future work of this project, I will adopt curriculum learning with progressively harder enemy configurations, attention mechanisms for variable numbers of enemies, and transfer learning to related game environments.

References

[1] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.

[2] Wang, Z., et al. (2016). Dueling Network Architectures for Deep Reinforcement Learning. ICML.

[3] Schaul, T., et al. (2016). Prioritized Experience Replay. ICLR.